

SituationReport

testdata_generator

Manuel d'utilisation

Générer des données de tickets Jira synthétiques pour SituationReport

situation-report -- github.com/Jaegerfeld/situation-report

Pour les Agile Coaches et les PI Managers -- Version 0.9.9

1 Qu'est-ce que le Générateur de données de test ?

Le **Générateur de données de test** (Testdata Generator) crée des données synthétiques de tickets Jira au format Jira REST API. Les fichiers générés peuvent être traités directement par le module **transform_data** — sans instance Jira réelle.

Le générateur simule des historiques de flux de travail réalistes : les tickets parcourent les étapes définies, passent des durées variables dans chaque étape, régressent occasionnellement et se ferment à un taux configurable.

Cas d'utilisation typiques :

- **Tester une nouvelle installation** : Vérifier que transform_data et build_reports fonctionnent correctement
- **Démos et présentations** : Données réalistes sans problèmes de confidentialité
- **Formation** : Environnement d'apprentissage contrôlé avec des données connues
- **Développement** : Données de test reproductibles via le paramètre seed

Le Générateur complète le module **Get Data** (en cours de développement), qui charge des données réelles directement depuis Jira. Pour l'export manuel de données Jira réelles, consultez le **Manuel d'utilisation Get Data**.

2 Prérequis et démarrage

2.1 Package portable (recommandé)

Le package portable inclut Python déjà intégré — aucune installation séparée requise.

Système d'exploitation	Fichier de démarrage
Windows	TestdataGenerator.bat (double-clic)
macOS	TestdataGenerator.command (clic droit → Ouvrir)
Linux	./TestdataGenerator.sh

Note Windows : Si un message d'erreur concernant Python apparaît, le fichier ZIP a peut-être été bloqué par Windows. Solution : clic droit sur le ZIP → Propriétés → Sécurité → Débloquer → OK → extraire à nouveau.

2.2 Depuis le code source (Python requis)

```
python -m testdata_generator
```

2.3 L'interface utilisateur

Au démarrage, la fenêtre principale affiche des champs de saisie pour tous les paramètres. Seul le fichier de flux de travail est obligatoire — tous les autres champs ont des valeurs par défaut sensées.

testdata_generator 0.9.2

Optionen / Options

Workflow-Datei

Ausgabedatei (JSON)

Projekt-Key

Anzahl Issues

Von (YYYY-MM-DD)

Bis (YYYY-MM-DD)

Issue-Typen (Typ:Gewicht ...)

Completion-Rate (0-1)

To-Do-Rate (0-1)

Backflow-Prob. (0-1)

Seed (leer = zufällig)

Log

Fig. 1 : Interface utilisateur du Générateur de données de test

3 Le fichier de flux de travail

Le fichier de flux de travail définit les étapes de votre tableau Jira et est le seul paramètre obligatoire. Il utilise le même format que `transform_data` — un seul fichier fonctionne pour les deux modules.

3.1 Format

Chaque ligne décrit une étape. Les alias (noms de statut alternatifs dans l'export Jira) sont séparés par des deux-points. Deux lignes spéciales marquent le début et la fin du travail actif :

```
NomCanonique:Alias1:Alias2
<First>PremiereEtapeActive
<Closed>EtapeFermee
```

Ligne	Signification
NomCanonique:Alias1:Alias2	Étape avec un ou plusieurs noms de statut Jira alternatifs
NomEtape (sans :)	Étape sans alias
<First>NomEtape	Première étape active — où commence le traitement (début du cycle time)
<Closed>NomEtape	Étape fermée — marque un ticket comme terminé

3.2 Exemple de flux ART_A

L'exemple fourni `workflow_ART_A.txt` modélise un flux SAFe ART typique.

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

4 Référence des paramètres

Tous les paramètres sont disponibles dans l'interface graphique et en ligne de commande.

Paramètre	Défaut	Description
--workflow FILE	(obligatoire)	Fichier de définition du flux de travail (.txt)
--output FILE.json	<projet>_generated.json	Chemin du fichier de sortie
--project KEY	TEST	Clé de projet Jira pour les tickets générés
--issues N	100	Nombre de tickets à générer
--from-date YYYY-MM-DD	2025-01-01	Date de création la plus ancienne
--to-date YYYY-MM-DD	2025-12-31	Date de transition la plus récente
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabling:0.1	Type(s) de tickets avec pondérations
--completion-rate FLOAT	0.7	Fraction de tickets atteignant l'étape fermée (0–1)
--todo-rate FLOAT	0.15	Fraction de tickets ouverts restant dans les étapes initiales (0–1)
--backflow-prob FLOAT	0.1	Probabilité d'une transition en arrière à chaque pas (0–1)
--seed INT	(aléatoire)	Seed pour résultat reproductible (optionnel)
--mean-cycle-days FLOAT	(aucun)	Cycle time moyen en jours (lognormal) ; active le contrôle CT
--std-cycle-days FLOAT	(30 % de la moyenne)	Écart type du cycle time en jours
--pattern MOTIF	none	Anti-pattern : none / triangle / flat_triangle / cluster / batch
--pi-duration-weeks INT	12	Durée du cycle PI en semaines (pour cluster/batch)

4b Modes d'anti-pattern de flux

Depuis la version 0.9.9, le Générateur peut simuler des anti-patterns de flux typiques de la littérature de Vacanti et Singh. Combinez **--mean-cycle-days** avec **--pattern**.

Motif	Apparence scatter plot	Description
none	Nuage de points uniforme	Cycle time aléatoire sans tendance — comportement par défaut
triangle	Triangle : angle droit en bas à gauche	Cycle time augmente linéairement dans le temps (multiplicateur 1–4×). Indique un problème de performance.
flat_triangle	Triangle, augmentation s'atténue à l'angle droit	Comme le triangle mais l'augmentation est atténuée par $\tanh(2.5 \times t)$. Montre une atténuation de la croissance.
cluster	Groupes verticaux de points en cycle time	Le cycle time reste stable, mais la plupart des tickets sont livrés dans les 2 derniers jours.
batch	Colonnes de hauteur variable	Comme le cluster mais avec cycle time très variable (0.1–3× moyenne). Montre que les tickets sont livrés en batch.

Contrôle du cycle time

```
# Motif triangle : CT augmente de 20j à 80j
python -m testdata_generator \
  --workflow workflow.txt --issues 300 \
  --pattern triangle \
  --mean-cycle-days 20 --std-cycle-days 6 \
  --completion-rate 0.8 --seed 1 \
  --output triangle.json
```

5 ART_A – Exemple complet

Cet exemple montre le pipeline complet du générateur au rapport final, en utilisant le projet fictif ART_A.

Étape 1 : Générer les données de test

```
python -m testdata_generator \  
    --workflow testdata_generator/workflow_ART_A.txt \  
    --project ART_A --issues 200 \  
    --from-date 2025-01-01 --to-date 2025-12-31 \  
    --completion-rate 0.75 --seed 42 \  
    --output ART_A_generated.json
```

Étape 2 : Traiter avec transform_data

```
python -m transform_data ART_A_generated.json \  
    --workflow testdata_generator/workflow_ART_A.txt
```

Étape 3 : Créer le rapport avec build_reports

```
python -m build_reports
```

6 Foire aux questions (FAQ)

Différence entre données de test et données réelles ?

Pour les démos, la formation et les tests d'installation, les données de test sont le meilleur choix — sans problèmes de confidentialité, reproductibles et entièrement contrôlées. Pour les analyses de flux réelles, les données réelles reflètent l'état réel du flux de travail.

Pour l'export manuel de données Jira réelles : Consultez le Manuel d'utilisation Get Data.

7 Glossaire

Terme	Explication
ART	Agile Release Train — structure multi-équipes dans SAFe
API	Application Programming Interface — interface de programmation
CFD	Cumulative Flow Diagram — comptage cumulé des tickets par étape
Cycle Time	Temps depuis la première étape active jusqu'à l'étape fermée
Issue	Élément de travail dans Jira (Feature, Bug, Enabler, Story, ...)
JSON	JavaScript Object Notation — format des exports API Jira
Seed	Valeur initiale pour le générateur aléatoire — même seed = même résultat
Stage	Une étape dans le flux de travail (correspond à une colonne Jira)
Workflow	Séquence ordonnée d'étapes que parcourt un ticket